PayFlow Events — Claude Code Prompt Runbook

Nov 7, 2026 · @gautam

Run these prompts in order in Claude Code, inside the payflow-events repo with its CLAUDE.md at the root. Each prompt is one commit and ends with a command that must pass. Start after Ledger prompt P1-04, because this pipeline reads the Ledger outbox schema.

Rules

1. **Setup once.** Put `CLAUDE.md` at the repo root. Export the Events plan doc as Markdown to `docs/plan.md`. Copy the Ledger outbox migration (`V5__outbox.sql`) to `docs/contract/outbox.sql`. Prompts refer to plan sections as "plan §N".
2. **One prompt = one commit.** Run `/clear` between prompts. CLAUDE.md, `docs/plan.md` and `docs/progress.md` carry the state.
3. **Plan first on big prompts.** Prompts tagged [plan] go in plan mode (Shift+Tab).
4. **The check decides.** If a Check fails, run X-01 before moving on.
5. **No weakening.** Reject any change that loosens an assertion, a correctness gate, `acks=all`, `min.insync.replicas` or the commit order to make something pass.
6. **Progress log.** Every prompt appends one line to `docs/progress.md`: prompt id, change, check result, RUN_IDs.
7. **Long runs.** Chaos and skew sweeps take hours. Start them with `make <target>`, which resumes after interruption, and run the next coding prompt in a second worktree.

Phase 1 prompts (weeks 1–2)

P1-01 Scaffold and Kafka stack [plan]

```
Read CLAUDE.md and docs/plan.md §2, §3 topics, §10 layout. Scaffold payflow-events as a multi-module Maven build (Java 25, Spring Boot 4.0.x): common, source-app, dual-write-app, relay-poller, consumer, rollup-streams, verifier, generator. Each app module has Jib.
deploy/docker-compose.yml: 3 Kafka 4.1 brokers in KRaft combined mode (RF=3, min.insync.replicas=2, auto.create.topics.enable=false), Apicurio Registry 3, postgres:18 (wal_level=logical, max_replication_slots=4), Kafka UI, toxiproxy, and a topic-init container that creates every topic in plan §3 with its exact partitions, retention and message.timestamp.type=LogAppendTime. Health checks gate order: brokers -> registry -> topic-init -> apps.
Makefile: up, down, test, it, chaos, skew, pills, rebalance, bench, report, clean. Unimplemented targets exit 1.
```

Check: `make up`, then `kafka-topics --describe` shows every topic with RF=3 and `LogAppendTime`.

P1-02 Telemetry stack as code [plan]

```
Read plan §4 and §6. Add deploy/observability/: otel-collector (contrib) with receivers otlp, kafkametrics (brokers, topics, consumers scrapers, 10 s), prometheus (scrape JMX exporter agents on kafka-1..3:7071 and connect:7071), postgresql, sqlquery (pg_replication_slot_lag_bytes, pg_outbox_backlog_rows, pg_outbox_oldest_unpublished_seconds as in plan §5), docker_stats. Processors memory_limiter, resource, batch, tail_sampling (ERROR 100%, >200 ms 100%, 2% rest, env override to 100%). Exporters to Prometheus 3 OTLP (promote service.name, run_id, scenario, relay), Tempo, Loki. Grafana datasources with exemplar and trace-to-logs links.
Add the Prometheus JMX exporter javaagent to brokers with deploy/observability/jmx/kafka.yml rules producing the broker metric names in plan §5. Pin every image.
```

Check: Grafana Explore shows `kafka_consumer_group_lag`, `kafka_server_replicamanager_underreplicatedpartitions` and `postgresql_backends`.

P1-03 common: contract, schema, metrics, migrations

```
Read plan §3 record shape and §5. In common/:
- Headers enum (id, type, seq, occurred_at, traceparent, merchant_id) and a
RecordShape helper that builds/reads a ProducerRecord from an outbox row.
- JSON Schema for each event type, registered in Apicurio with BACKWARD
compatibility by scripts/register-schemas.sh; CI step that fails on an
incompatible change.
- EventsMetrics: the only place meters are created, names and labels exactly
as plan §5.
- application-telemetry.yml shared by all apps: OTLP endpoints, resource
attributes from RUN_ID/SCENARIO/RELAY, SLO buckets from plan §4, Kafka
observation on. OtelLogbackInstaller bean.
- Flyway: V1 outbox copied byte-for-byte from docs/contract/outbox.sql, V2
payments (source business table), V3 processed_events partitioned by day +
partition maintenance function, V4 aggregate_seq(aggregate_id, last_seq).
Tests: RecordShape round-trip, migration test on postgres:18, a test that V1
equals docs/contract/outbox.sql.
```

Check: `make test it` green; the schema-compat script rejects a deliberately broken schema.

P1-04 source-app and dual-write-app

```
source-app: writes one payments row and its outbox row
(aggregate_type=payment, seq = payment version, traceparent from the current
span) in one transaction, at a fixed rate from env RATE, with
outbox.mode=retain|delete. dual-write-app: same business write, then two
modes: db-first (commit, then producer.send) and kafka-first (send and wait
for ack, then commit; force a rollback on 0.1% of events). Both record their
committed event ids in the business table so the verifier can read them. Both
use virtual threads and report payflow_generator_sent_total.
```

Check: both apps sustain 5k events/s for 60 s (Grafana panel), Postgres CPU visible.

P1-05 relay-poller + contract test

```
Read plan §3 Relay A. Implement relay-poller: SELECT ... FOR UPDATE SKIP
LOCKED LIMIT ${BATCH:500}, send with the idempotent producer (acks=all,
enable.idempotence=true, compression zstd), wait for all acks, UPDATE
published_at, commit. Idle backoff 5 ms / 50 ms. Metrics
payflow_relay_batch_size, _poll_seconds, _published_total. Run 2 instances to
prove no double-pick.
Contract test (Testcontainers Kafka + Postgres): one outbox row -> record
with exact key, value bytes and headers from plan §3.
```

Check: `make it` green; with 2 relay instances, duplicates received in a 100k run = 0 without crashes.

P1-06 Idempotent consumer [plan]

```
Read plan §3 Consumer. Implement consumer with Spring Kafka, manual ack,
auto-commit off:
- Phase 1 baseline: per-event INSERT INTO processed_events ... ON CONFLICT DO
NOTHING in one DB transaction with the side effect (update a merchant_balance
table) and the seq check against aggregate_seq (gap or reversal -> count
payflow_events_order_violations_total and log with trace id; do not apply a
reversal).
- Ack offsets only after the DB commit.
- Record e2e, relay and consume latency per plan §4 (t_occurred header,
record timestamp, now after commit), consumer_lag_seconds per partition,
processed_total{result}.
- Kafka observation continues the trace from the traceparent header.
Tests: re-publish the same 10k events 3 times -> 10k effects, 20k duplicates
counted; 1k payments x 5 events with random consumer restarts -> 0 order
violations.
```

Check: `make it` green; a Tempo trace shows source-app -> relay -> consumer spans in one trace.

P1-07 Verifier

```
Implement verifier as a CLI and a scheduled job: for a RUN_ID, compare
committed source event ids (business table), Kafka (read payment.events from
the run's start offsets), and processed_events. Output lost, phantom,
duplicate_effects, duplicates_received, order_violations, dlq_count,
pills_injected, rollup_mismatch. Export each as the payflow_verifier_* gauges
with run_id, scenario, relay labels, plus last_run_timestamp; append a row to
docs/results.json. Use set operations in SQL or sorted-merge streams; never
load 1M ids into a HashSet per side if a SQL anti-join works.
Tests: seeded fixtures for each failure kind are detected exactly.
```

Check: `make it` green; verifier on a clean 100k run prints all zeros.

P1-08 Crash test v1

```
generator: uniform keys, fixed seed, rate and total from env. chaos/kill-writer.sh: kill -9 the writer container every 10-30 s (random, seeded), restart it, post a Grafana annotation. make chaos-v1 runs 1M events three times: dual-write db-first, dual-write kafka-first, source-app + relay-poller. Each run sets RUN_ID/SCENARIO/RELAY and ends with the verifier.
```

Check: results.json has three rows: db-first lost > 0, kafka-first phantom > 0, poller all zeros.

P1-09 Dashboards, alerts, report

```
Read plan §6. Provision dashboards Pipeline Overview, Latency Breakdown, Relays and Correctness as JSON; recording rules events.yml exactly as written; alerts LostEvents, DuplicateEffects, OrderViolations, VerifierStale, OutboxBacklogOld. perf/report.py: for each RUN_ID in results.json, query the Prometheus API over the run's measure window and write docs/results.md tables (correctness, latency split, throughput, CPU) with the machine spec.
```

Check: `make report` regenerates results.md with no hand-typed numbers.

P1-10 Baseline, CI, Phase 1 gate

```
Run 1M uniform events through source-app + relay-poller at 1k and 5k events/s, telemetry on and off. Add .github/workflows/ci.yml: build, unit, schema compatibility, contract test, Testcontainers ITs (idempotency, ordering), JaCoCo gate, Jib images to GHCR on main. Then check plan §7 exit gate item by item and record evidence in docs/progress.md.
```

Check: CI green; every Phase 1 exit-gate box has evidence.

Phase 2 prompts (weeks 3–4)

P2-01 Debezium CDC relay [plan]

```
Read plan §3 Relay B. Add Kafka Connect (Debezium 3.4+ Postgres connector
image) to compose with the JMX exporter agent
(deploy/observability/jmx/connect.yml producing debezium_metrics_* names from
plan §5). connect/debezium-outbox.json exactly as plan §3;
connect/register.sh registers it via the REST API after Connect is healthy,
creating the publication for public.outbox only. Verify every connector and
SMT key against Debezium 3.4 docs and fix any that differ. Run source-app
with outbox.mode=delete. Run the contract test against the CDC relay: record
must match the poller's byte-for-byte. Testcontainers CDC test: 10k rows
inserted and deleted -> 10k records.
```

Check: contract test passes for both relays; `select count(*) from outbox` stays under 100 during a 5k/s run.

P2-02 Polling vs CDC benchmark

```
Run source-app at 5k events/s and then at a step-up rate until consumer lag
grows (max sustained), once with relay-poller (outbox.mode=retain) and once
with CDC (delete), 1M events each, verified. Report e2e p99 split into relay
and consume, events/s, Postgres CPU, slot lag max, poller backlog max. Add
the table and docs/polling-vs-cdc.png (bars per metric) to results.md via
report.py.
```

Check: results.md has the polling-vs-CDC table with RUN_IDs; both runs verified at 0/0/0/0.

P2-03 Chaos matrix [plan]

```
Read plan §8 chaos matrix. chaos/: kill-writer.sh, kill-broker.sh (docker
kill -s KILL one broker, restart after 60 s), restart-postgres.sh, restart-
connect.sh, kill-consumer.sh (kill -9 while a batch is mid-transaction:
trigger on a log line), toxiproxy-consumer-db.sh (100 ms latency +
reset_peer). make chaos runs every scenario x relay pair from the table, 1M
events each, with tail sampling at 100%, resets state per run (plan §8 run
hygiene), runs the verifier, prints a PASS/FAIL table and exits non-zero on
any outbox FAIL. Resumable via a state file.
```

Check: `make chaos` prints PASS for every outbox row; dual-write rows show counted losses/phantoms.

P2-04 Two-stage rollups with exactly_once_v2

```
Read plan §3 skew strategy. rollup-streams: stage 1 reads payment.events, re-keys to merchant_id#(murmur2(payment_id) % B), sums amount per 1 s tumbling window to merchant.rollup.stage1; stage 2 re-keys by merchant_id and sums the B partials into compacted merchant.totals.
processing.guarantee=exactly_once_v2, classic rebalance protocol, B from env (default 8). verifier computes payflow_rollup_mismatch_minor = sum(merchant.totals) - source sum per merchant. Test with a hot merchant at 40% and a Streams instance kill.
```

Check: rollup mismatch = 0 after the kill test.

P2-05 Skew lab [plan]

```
Read plan §8 days 15-16. generator: Zipf mode with one merchant at 40% of events (fixed seed), report payflow_generator_hot_share. Key strategy switch for the experiment: merchant_id, aggregate_id (payment_id), salted two-stage.
perf/skew-matrix.sh: 3 strategies x partitions {6,12,24,48} (recreate topic per run) x consumers {1, P/4, P/2, P} at a fixed rate, 3 min each, 60 s warm-up excluded, resumable. Record events/s, e2e p99, max lag seconds, partition imbalance. report.py renders docs/skew.png (p99 and imbalance vs partitions, one line per strategy) and a matrix table.
```

Check: `make skew MATRIX=smoke` (2 runs) works; the full run fills the table; imbalance < 1.5 for aggregate_id key.

P2-06 Spike test

```
At the best config from P2-05, run baseline rate for 2 min, then 10x for 60 s, then baseline for 5 min. Measure peak events:max_lag_seconds and the time until lag < 1 s. Add both numbers and a lag-over-time PNG to results.md.
```

Check: results.md shows peak lag and recovery time with the RUN_ID.

P2-07 Poison pills and DLQ replay

```
Read plan §3 consumer step 4. Configure retries with
RetryTopicConfigurationBuilder (or @RetryableTopic): attempts 3,
@Backoff(delay = 5000, multiplier = 12) so the retry topics are 5 s and 1 m,
autoCreateTopics false, a topic-naming strategy that yields payment.events-
retry-5s / -retry-1m and the DLT payments.dlq (if custom DLT naming fights
the framework, rename the DLT to payment.events-dlq in plan §3 and topic-init
instead) with non-retryable classification for deserialization and validation
errors (straight to DLQ). generator injects 0.1% pills (malformed payload or
a flag that makes the side effect throw), counting
payflow_verifier_pills_injected. Label e2e latency poison=true|false.
consumer/dlq-replay CLI: reads payments.dlq with group dlq-replay, re-
publishes fixed events to payment.events keeping id and headers, counts
payflow_events_dlq_replayed_total. make pills runs with and without pills and
compares healthy p99.
```

Check: DLQ count = injected; healthy p99 with pills within 10% of without; replay drains `dlq-replay` lag to 0.

P2-08 Rebalance benchmark

```
Run 4 consumer instances at 5k events/s and do a rolling restart (one at a
time, 20 s apart) under three configs: classic + RangeAssignor (eager),
classic + CooperativeStickyAssignor + group.instance.id (static),
group.protocol=consumer (KIP-848). Measure max
payflow_events_consumption_gap_seconds, total rebalance time, duplicates
received, and e2e p99 during the restart window. Add a table and chart to
results.md. Correctness must stay 0/0/0/0.
```

Check: rebalance table has three rows with RUN_IDs.

P2-09 Tuning step (run once per step T1–T6)

```
Tuning step <T#> from plan §8: <paste the row>.
1. Record BEFORE RUN_IDs for 5k events/s uniform and 40% skew at the current
best config.
2. Implement only this change behind config. For sweeps, run each value.
3. make test it, then bench both scenarios as AFTER and run the verifier.
4. Keep only if correctness is 0 and events/s or p99 improves >= 5% without
the other getting >5% worse. Otherwise revert.
5. Add a row to the results.md tuning table with the metric that explains the
change.
```

Check: tuning table row exists with both RUN_IDs.

P2-10 SLOs and burn-rate alerts

```
Read plan §6. Add the remaining recording rules (SLI over 5m, 30m, 1h, 6h)
and alerts ConsumerLagHigh, LatencyFastBurn, DLQReceiving, DLQNotReplayed,
DebeziumBehind, ReplicationSlotLag, UnderReplicated exactly as written.
Provision dashboards Partitions & Skew, Consumers & Rebalance, Brokers.
promtool unit tests: one firing and one non-firing case per alert.
```

Check: `promtool test rules` passes; seven dashboards load with no errors.

P2-11 Nightly pipeline

```
Add .github/workflows/nightly.yml: compose up, make chaos (subset: writer
kill, broker kill, consumer kill for both relays), make pills, CDC
Testcontainers test, skew smoke, make report. Upload results.json,
results.md, PNGs and Grafana renders. Fail on any outbox correctness FAIL.
Keep it under 90 min with Docker layer caching.
```

Check: manual `workflow_dispatch` run green with artifacts.

P2-12 Ledger integration

```
Add a compose profile ledger that runs the payflow-ledger image (from GHCR)
against the same Postgres instead of source-app, with
ledger.outbox.mode=delete and CDC. Drive it with the Ledger's Gatling
TransferSimulation at 200 users. Verify 0 lost / 0 duplicate effects, and
capture one Tempo trace from POST /v1/transfers to the consumer commit as
docs/trace.png.
```

Check: verifier all zeros on the ledger profile; trace screenshot saved.

Finish and release

F-01 README and results

```
Write README.md: problem in one paragraph, architecture diagram
(docs/architecture.png from a Mermaid source), how to run (make up, URLs for
Grafana, Kafka UI, registry), the dual-write vs outbox chaos table as the
headline, polling-vs-CDC chart, latency split, skew chart, spike result,
poison-pill result, rebalance table, tuning table, the end-to-end trace
screenshot, machine spec, and the exact make command behind every number. DLQ
replay CLI usage with an example. No number that is not in docs/results.json.
```

Check: every number in README.md exists in docs/results.json.

F-02 Kubernetes deploy (optional)

```
Add deploy/strimzi/ (Kafka 4.1 KRaft cluster, KafkaConnect with the Debezium plugin, KafkaConnector, KafkaTopic resources matching plan §3) and deploy/helm/ charts for consumer, relay-poller, rollup-streams. KEDA ScaledObject scaling consumers on lag (max = partitions). OTel Collector via its Helm chart. scripts/kind-up.sh brings it all up in kind; scripts/kind-smoke.sh runs 10k events and the verifier.
```

Check: `scripts/kind-up.sh && scripts/kind-smoke.sh` prints all zeros.

F-03 Fresh-clone audit

```
Clone into /tmp/fresh, run make up, make test it, make chaos MATRIX=smoke, make skew MATRIX=smoke, make pills, make report. Walk plan §10 definition of done line by line, mark each with evidence in docs/progress.md, and fix every gap in a separate commit.
```

Check: all definition-of-done boxes ticked with evidence.

F-04 Release

```
Tag v1.0.0, push Jib images for relay-poller, consumer, rollup-streams, verifier and generator tagged with the SHA and 1.0.0, publish connect/debezium-outbox.json and results.json as release assets with notes from docs/progress.md.
```

Check: release page lists all five images and the assets.

Fix-loop prompts (use any time)

X-01 A check failed

```
The check for <prompt id> failed: <paste output>. Reproduce it with the smallest failing test, state the root cause in one sentence, then fix it. Do not loosen acks, min.insync.replicas, commit order, assertions or correctness gates. Run the full check again and paste the result.
```

X-02 Lost, phantom, duplicate or reordered event

```
Verifier for <RUN_ID> reports <metric> = <n>. List the offending event ids. For each, find: the outbox row (or its absence), the Kafka record (topic, partition, offset, headers), the processed_events row, and the Tempo trace by traceparent; check the chaos annotations around that time. Write the exact sequence of events that caused it, add a deterministic Testcontainers test that reproduces it, then fix. Re-run the failing chaos scenario.
```

X-03 Latency or throughput regression

```
<metric> got worse from <RUN_ID_A> to <RUN_ID_B>. Via the Prometheus API compare: relay p99 vs consume p99, poller poll time or Debezium ms behind, producer batch size, record queue time and compression ratio, consumer records-lag-max and fetch latency, dedup insert p99, Postgres CPU and wait events, broker request handler idle %, partition imbalance. Open 3 exemplar traces at run B's p99 and list where the time went. Name the cause with numbers, propose one change, and measure it with P2-09.
```

X-04 Telemetry is missing

```
<metric or span> is not in Grafana. Check each hop and stop at the first broken one: meter/span created (unit test), JMX MBean exists (jmxterm) for broker/Connect metrics, exporter rule matches (curl the agent on :7071), app OTLP export (exporter debug log), Collector receive and send (otelcol_receiver_accepted_*, otelcol_exporter_sent_*, send_failed_*), store (Prometheus/Tempo/Loki API), dashboard query and label names. Fix that hop only.
```

X-05 Debezium slot or connector stuck

```
DebeziumBehind or ReplicationSlotLag is firing. Check: connector and task status via the Connect REST API, the task's last error, pg_replication_slots (active, restart_lsn, confirmed_flush_lsn), publication membership of public.outbox, heartbeat events flowing. Fix the cause, restart only the task if needed, and confirm with a 100k-event verified run that nothing was lost or applied twice.
```

X-06 Context reset

```
Summarize the current state into docs/progress.md: last completed prompt id, open failures, decisions and why, numbers so far with RUN_IDs, any running sweeps and their state files. Under 40 lines. Then stop.
```